NEXT-GEN ROGUE DEVICE DETECTION SYSTEM

A PROJECT REPORT

Submitted by

SUDHARSAN S (2116231901054)

PREM ROSHAN P (2116231901036)

in partial fulfilment for the award of the degree

of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING (CYBER SECURITY)





RAJALAKSHMI ENGINEERING COLLEGE:CHENNAI 602 105

ANNA UNIVERSITY: CHENNAI 600 025

OCTOBER 2026

RAJALAKSHMI ENGINEERING COLLEGE

CHENNAI

BONAFIDE CERTIFICATE

Certified that this Project report titled "**NEXT-GEN ROGUE DEVICE DETECTION SYSTEM**" is the bonafide work of "**SUDHARSAN S (231901054), PREM ROSHAN P (231901036)**" who carried out the work under my supervision. Certified further that to the best of my knowledge the work reported herein does not form part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE

Dr. P. Tamilselvi, Ph.D.,

PROFESSOR & HEAD OF THE DEPARTMENT

Department of Computer Science and Engineering (Cyber Security)

Rajalakshmi Engineering College
Chennai – 602 105

SIGNATURE

Ms. Gayathripriya P.S

ASSISTANT PROFESSOR

Department of Computer Science and Engineering (Cyber Security)

Rajalakshmi Engineering College
Chennai – 602 105

Submitted to Project Viva-Voce Examination held on _____

Internal Examiner **External Examiner**

ABSTRACT

The rapid growth of Bring Your Own Device (BYOD), Internet-of-Things (IoT) endpoints and unmanaged peripherals has weakened the assumption that a corporate local-area network contains only known and trusted assets. Identity-based controls such as MAC-address whitelisting are inexpensive but limited, because a network-visible identifier is not cryptographic proof of device identity. This project measured that weakness directly: on a controlled 25-event workload with 20 legitimate and 5 MAC-spoofed events, the MAC-only baseline produced TP=0, FP=0, TN=20 and FN=5, so precision, recall and F1-score were all 0.00 even though accuracy was 80 %. The report reframes this result within an anomaly-based rogue-device detection framework that combines identity, switch-port history, DHCP Option-55 fingerprints, traffic and temporal behaviour, device profiles and an interpretable, reason-coded risk score. Rule-based multi-signal fusion is the primary implementation path; Isolation Forest, Local Outlier Factor and One-Class SVM are specified as comparative models once enough legitimate history exists. The evaluation methodology uses adversary tiers T1–T3 and reports recall, precision, F1-score, false-positive rate and detection latency. Completed evidence and planned experiments are kept strictly separate throughout.

ACKNOWLEDGEMENT

First, we thank the almighty God for the successful completion of the project. Our sincere thanks to our Chairman **Mr. S. Meganathan B.E., F.I.E.,** for his sincere endeavor in educating us in his premier institution. We would like to express our deep gratitude to our beloved Chairperson **Dr. Thangam Meganathan Ph.D.,** for her enthusiastic motivation which inspired us a lot in completing this project and Vice Chairman **Mr. Abhay Shankar Meganathan B.E., M.S.,** for providing us with the requisite infrastructure.

We also express our sincere gratitude to our college Principal **Dr. S.N. Murugesan M.E., Ph.D.,** and **Dr. P. Tamilselvi Ph.D., Professor and Head, Department of Computer Science and Engineering (Cyber Security)** and our project guide **Ms. Gayathripriya P.S, Assistant professor,** for his/her encouragement and guiding us throughout the project. We would like to thank our project coordinator **Mr. Benedict J.N., M.E., Assistant Professor (SG),** for his encouragement and guidance towards successful completion of this project and to our parents, friends, all faculty members and supporting staffs for their direct and indirect involvement in successful completion of the project for their encouragement and support.

SUDHARSAN S

PREM ROSHAN P

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	**iii**
	LIST OF TABLES	**vii**
	LIST OF FIGURES	**viii**
1	**INTRODUCTION**	**1**
	1.1 BACKGROUND	1
	1.2 PROBLEM STATEMENT	2
	1.3 OBJECTIVES	2
	1.4 SCOPE	3
	1.5 SECURITY REQUIREMENTS / SECURITY CONSIDERATIONS	3
	1.6 SDG ALIGNMENT	4
2	**LITERATURE REVIEW**	**5**
	2.1 EXISTING SYSTEM WITH SECURITY APPROACHES	5
	2.2 RESEARCH GAPS	6
	2.3 PROPOSED SYSTEM	7
	2.3.1 OVERVIEW OF THE PROPOSED SYSTEM	7
	2.3.2 OBJECTIVES OF THE PROPOSED SYSTEM	7
	2.3.3 ADVANTAGES OF PROPOSED SYSTEM	8
3	**SYSTEM DESIGN**	**9**
	3.1 SYSTEM ARCHITECTURE	9
	3.2 MODULE DESIGN	10
	3.3 DATA FLOW / UML DIAGRAMS	13
	3.4 THREAT MODEL AND ATTACK SURFACE	16
	3.5 SECURITY CONTROLS	18
4	**SYSTEM DESCRIPTION**	**20**
	4.1 DATA COLLECTION	20
	4.2 DATA PREPROCESSING	21

CHAPTER NO.	TITLE	PAGE NO.
	4.2.1 ALGORITHMS / METHODOLOGY	22
	4.2.2 RESULTS AND DISCUSSION	23
5	**CONCLUSION**	**26**
	REFERENCES	**27**
	APPENDIX	**29**
	A. SOURCE CODE OF THE PHASE III SLICE	29
	B. LIST OF ABBREVIATIONS	31

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
1.1	SECURITY REQUIREMENTS OF THE DETECTION FRAMEWORK	3
1.2	SDG ALIGNMENT	4
2.1	COMPARISON OF DEVICE-IDENTIFICATION AND DETECTION APPROACHES	6
2.2	SUMMARY OF REVIEWED LITERATURE	6
2.3	PROPOSED SYSTEM COMPARED WITH THE EXISTING MAC-WHITELIST CONTROL	8
3.1	HARDWARE REQUIREMENTS (REFERENCE CONFIGURATION)	10
3.2	SOFTWARE REQUIREMENTS	10
3.3	MODULE INTERFACES AND IMPLEMENTATION STATUS	11
3.4	WORKED EXAMPLES OF RISK SCORING WITH POLICY FLOORS (ILLUSTRATIVE)	12
3.5	THREAT ENUMERATION (STRIDE CHECKLIST)	18
3.6	SECURITY CONTROLS AND HOW THEY ARE VERIFIED	18
4.1	EVENT DATASET SCHEMA	20
4.2	COMPOSITION OF THE PHASE III WORKLOAD (FROM THE EXECUTED LOG)	21
4.3	DETECTOR CONFIGURATIONS FOR THE ABLATION STUDY AND RESULT TEMPLATE	24
B.1	COMPLETE EVENT LOG OF THE PHASE III WORKLOAD (SORTED BY TIME)	32

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
1.1	IDENTICAL MAC, DIFFERENT EVIDENCE: WHY A MAC-ONLY DECISION FAILS	2
2.1	OVERVIEW OF THE PROPOSED ANOMALY-BASED ROGUE-DEVICE DETECTION SYSTEM	7
3.1	LAYERED SYSTEM ARCHITECTURE OF THE PROPOSED DETECTOR	9
3.2	MODULE DECOMPOSITION AND INTERFACES	10
3.3	RISK SCORE MAPPED TO OPERATIONAL STATES WITH WORKED EXAMPLES	12
3.4	LEVEL-1 DATA FLOW DIAGRAM	13
3.5	USE CASE DIAGRAM	14
3.6	ACTIVITY DIAGRAM OF THE DETECTION PROCEDURE FOR ONE OBSERVATION	15
3.7	SEQUENCE DIAGRAM FOR PROCESSING ONE DEVICE OBSERVATION	16
3.8	TRUST ZONES AND TRUST BOUNDARIES (TB1 TO TB5)	17
3.9	ADVERSARY TIERS AND THE EVIDENCE EACH TIER REPRODUCES	17
4.1	PHASE III WORKLOAD: SWITCH PORT OF EACH EVENT OVER TIME AND THE DECISION THE BASELINE GAVE	21
4.2	FEATURE TAXONOMY USED BY THE FRAMEWORK	22
4.3	MEASURED CONFUSION MATRIX OF THE MAC-WHITELIST BASELINE (N = 25)	23
4.4	FIELDS THAT DIFFER FROM THE REGISTERED PROFILE IN EACH SPOOFED EVENT (EXECUTED LOG)	24

CHAPTER 1
INTRODUCTION

1.1 BACKGROUND

Corporate networks no longer contain only known, managed assets. Bring Your Own Device (BYOD) practices, Internet-of-Things (IoT) endpoints, unmanaged peripherals and wireless bridges mean that the question for a defender has shifted from "did this packet cross a permitted boundary?" to "which device generated this traffic, where is it connected, and is its behaviour consistent with its role?" This is the central premise of zero-trust thinking, which treats no network location as implicitly trusted [1]. Device visibility is therefore a prerequisite for access control, and an asset inventory alone is not enough, because the observed network state can differ from the intended one.

A rogue device is defined in this report as an unauthorized device present on the monitored network. The definition does not require malicious intent: a personal laptop, a consumer router, an unmanaged camera, a phone hotspot or a development board can each violate policy before any attack is attempted. The same condition can also be created deliberately, for example by plugging a laptop into an exposed port, installing a small access point, or impersonating a trusted device identifier.

The simplest existing control is a static identity list such as a MAC allow-list. It is cheap and explainable, but a MAC address is a self-reported value: it can be changed in software, and the Address Resolution Protocol that binds it to an IP address has no authentication [2]. An IP address is normally leased dynamically by DHCP [3], and a hostname can be edited freely. A pure allow-list also has no behavioural context, so an approved identity that suddenly appears on an unexpected port, with a different protocol fingerprint, is still accepted.

This weakness was measured rather than assumed. In the Phase III prototype, a MAC-whitelist detector was run over 20 legitimate and 5 MAC-spoofed events, and it accepted all five rogue events because the presented MAC belonged to the whitelist (Figure 1.1). The failure is structural: if a detector reads only a field the attacker can reproduce exactly, no threshold can recover information that is absent from its input.

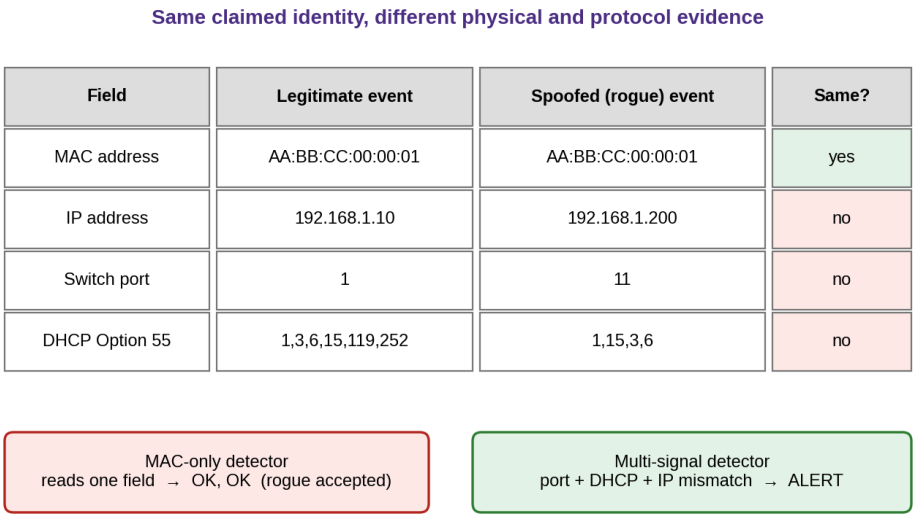


Figure 1.1 Identical MAC, different evidence: why a MAC-only decision fails

Anomaly-based detection addresses this by comparing each observation with a profile of what is normal for that device, using identity, switch-port history, DHCP fingerprint, traffic and timing together [4]. Deterministic evidence is interpretable, while statistical evidence can reveal subtle deviations that are difficult to write as rules. Because legitimate networks also change, the framework uses graded states (OK, REVIEW, ALERT), grace periods and controlled profile updates, and treats the detector as decision support in which the administrator keeps the final authority.

1.2 PROBLEM STATEMENT

Corporate environments need continuous identification of the devices on their networks. Low-cost controls often rely on static identifiers that an attacker can reproduce, while purely machine-learning approaches need representative datasets and continuous maintenance. The problem addressed here is to design and evaluate a practical anomaly-based framework that combines several network and behavioural signals to identify rogue devices, while keeping every decision interpretable and measurable. The Phase III evidence shows that the MAC-only baseline produced zero true positives; this report extends it with network location, DHCP fingerprinting, behavioural history and anomaly scores.

1.3 OBJECTIVES

The aim is to design, specify and evaluate an interpretable anomaly-based rogue-device detection framework for corporate LANs that remains useful when an adversary reproduces individual identity signals. The specific objectives are:

1. To study rogue-device risks and the limits of identity-only controls.

2. To identify network and behavioural signals that can be collected without payload inspection.
3. To design a device-profile representation that supports controlled learning and explainable comparison.
4. To design a multi-signal architecture with a transparent rule layer and an optional unsupervised anomaly layer.
5. To implement a reproducible baseline (the Phase III slice) that generates labelled events, logs decisions and computes metrics.
6. To define adversary tiers T1 to T3 and evaluate precision, recall, F1-score, false-positive rate and detection latency.

1.4 SCOPE

The study covers local and enterprise-style LANs in which passive telemetry (MAC, IP, switch-port history, DHCP Option-55 fingerprint, timestamps and traffic summaries) can be collected. The scenarios are unauthorized laptops, unmanaged IoT devices, rogue access points, personal hotspots and MAC-spoofed devices. Out of scope are compromised but authorized hosts (an intrusion-detection problem), payload inspection, radio-layer fingerprinting, automatic blocking and production SIEM deployment. Cryptographic admission control such as IEEE 802.1X [5] is not replaced; the framework is a complementary visibility layer.

The report separates what has been measured from what is proposed. Only the synthetic MAC-spoofing baseline on 25 events was executed; the fusion engine, profile manager and anomaly engine are specified in detail and are reported as "to be measured". Illustrative figures and worked examples are labelled as such and are never presented as experimental findings.

1.5 SECURITY REQUIREMENTS / SECURITY CONSIDERATIONS

A detector that collects network metadata must itself be designed securely. The requirements in Table 1.1 follow the confidentiality, integrity and availability triad, with added requirements for explainability and auditability, and are traced to the controls of Section 3.5.

Table 1.1 Security requirements of the detection framework

ID	Requirement	Addressed in
SR-1	Detect unauthorized presence using more than one independent signal	Sections 3.2, 4.2.1
SR-2	Keep a reason code for every REVIEW or ALERT decision	Sections 3.2, 3.5
SR-3	Protect profile and log integrity against tampering and poisoning	Section 3.5
SR-4	Update profiles only from confirmed legitimate observations	Sections 3.2, 3.5
SR-5	Use metadata only, without payload decryption	Sections 4.1, 3.5
SR-6	Fail safe: missing telemetry is never read as proof of legitimacy	Sections 4.2, 3.5
SR-7	Run passively with least privilege; no automatic blocking in the prototype	Section 3.5

Privacy is handled by limiting collection to metadata, restricting log access and defining retention periods. Spoofing and rogue-access-point experiments are performed only in an isolated emulated or laboratory network.

1.6 SDG ALIGNMENT

The project supports the United Nations Sustainable Development Goals [6], chiefly SDG 9 (Industry, Innovation and Infrastructure) and, indirectly, SDG 11 (Sustainable Cities and Communities), because colleges, hospitals and smart buildings increasingly depend on networks that mix managed and unmanaged devices.

Table 1.2 SDG alignment

S. No.	SDG No. & Goal	Relevant SDG Target	Project Contribution / Alignment
1	SDG 9: Industry, Innovation and Infrastructure	9.1: Resilient infrastructure	Continuous visibility of unauthorized devices improves the reliability of enterprise network infrastructure
2	SDG 9: Industry, Innovation and Infrastructure	9.5: Scientific research and technological capability	A reproducible methodology (adversary tiers, ablation, honest reporting) that others can extend
3	SDG 11: Sustainable Cities and Communities	11.b: Resilience (indirect)	Strengthens the cyber-resilience of IoT-heavy campuses and smart buildings

CHAPTER 2
LITERATURE REVIEW

2.1 EXISTING SYSTEM WITH SECURITY APPROACHES

Enterprise security is implemented through overlapping controls: firewalls, VLAN segmentation, authentication, network access control, endpoint protection and centralized logging [7]. Rogue-device detection sits in the visibility layer. It looks for discrepancies between the intended inventory and the actual device population, and it provides an independent observation channel when preventive controls are misconfigured or bypassed. This report relies on metadata (DHCP, ARP, switch-port and flow summaries) because it can be collected without installing software on every endpoint and has a smaller privacy impact than payload inspection.

Identity-based controls. MAC allow-lists and switch port security need no training data and run at line rate, which is why they remain common. Their weakness is that the identifier is self-reported: the address can be changed in software and ARP is unauthenticated [2]. The verdict is also binary, with no representation of confidence, context or behaviour.

IEEE 802.1X. Port-based access control [5] authenticates a supplicant through the Extensible Authentication Protocol [8] against a RADIUS server [9], so a laptop that merely copies a MAC address is not admitted. The limits are operational: certificates must be provisioned, headless devices such as printers and cameras often fall back to MAC authentication bypass, and an authenticated port can later be extended with an unmanaged switch. Anomaly detection is therefore treated as complementary to 802.1X.

Device fingerprinting. A fingerprint combines observable protocol behaviour such as the DHCP Option 55 parameter request list [10], whose ordering differs between client implementations and can be seen passively [3]. IoT Sentinel identified device types from traffic seen at first connection [11], Sivanathan et al. classified IoT devices from traffic characteristics [12], and Bezawada et al. used behavioural fingerprints [13]; a survey notes inconsistent evaluation practice [14]. These studies classify devices into known types and give probabilistic answers, so they are used here as a source of features and not as proof of identity.

Behavioural anomaly detection. Chandola et al. give the standard taxonomy of anomaly-detection techniques [4]. N-BaIoT trains autoencoders on benign IoT traffic [15] and Kitsune uses an ensemble of small autoencoders [16], but both target malicious behaviour, whereas a rogue device can behave normally and still violate policy. Three unsupervised

models are specified for comparison: Isolation Forest, which isolates points by random partitioning [17], Local Outlier Factor, which compares local density [18], and One-Class SVM, which learns a boundary around normal data [19], all available in scikit-learn [20]. Table 2.1 compares the families and Table 2.2 summarizes the studies most relevant to this work.

Table 2.1 Comparison of device-identification and detection approaches

Approach	Needs labels	Explainability	Main weakness
MAC allow-list	No	High	Forgeable identifier
DHCP fingerprint	No	High	Mutable, probabilistic
Supervised ML	Yes	Variable	Novel devices unseen
Isolation Forest / LOF / One-Class SVM	No (normal data only)	Medium	Threshold and parameter sensitivity
IEEE 802.1X	Credentials	High at policy layer	Provisioning cost and exceptions

Table 2.2 Summary of reviewed literature

Study	Method	Limitation for this project
Miettinen et al. [11]	Early-connection device fingerprinting	Known device types; not identity impersonation
Sivanathan et al. [12]	Traffic-characteristic classification	Fixed known device population
Meidan et al. [15]	Deep autoencoder, benign-only training	Detects malicious traffic, not unauthorized presence
Alani et al. [21]	Explainable deep learning for ARP spoofing	Not centred on adaptive identity spoofing
Wang et al. [22]	Supervised ML on a small IoT testbed	Lower performance at realistic scale
Lantz et al. [23]	Mininet network emulation	Emulation differs from production

The existing approach used as the control condition is a MAC-whitelist detector holding three known MAC addresses: a listed address gives OK and any other gives ALERT. It is intentionally minimal so that its behaviour can be attributed to the identity-only mechanism, and every improvement in this project is reported relative to it.

2.2 RESEARCH GAPS

The literature separates device-type identification, malicious-traffic detection, network admission control and unauthorized-presence detection, but these objectives overlap without

being identical: a classifier can recognize a laptop without knowing whether it is the authorized one, and an intrusion detector can miss a benign but unauthorized device. Four gaps motivate the work.

1. **Identity versus device type.** Fingerprinting studies classify known types; identity consistency has to be modelled separately.
2. **Limited adversarial evaluation.** Few evaluations ask what happens when the attacker knows which feature is used and imitates it.
3. **Single-signal fragility.** Each signal can be forged or can drift; integrated, interpretable combinations of identity, port, fingerprint and behaviour are less represented.
4. **Operational explainability.** An alert saying "score = 0.91" is much less useful than one naming the new port, the unfamiliar fingerprint and the unusual destination count.

2.3 PROPOSED SYSTEM

2.3.1 OVERVIEW OF THE PROPOSED SYSTEM

The proposed system observes devices continuously and builds a profile for each authorized identity (known MACs, expected ports, DHCP fingerprints and behavioural ranges). Each observation becomes a feature vector. A rule-based fusion layer tests explicit contradictions with the profile, an optional anomaly layer measures statistical deviation, and a risk scorer maps the combined evidence to OK, REVIEW or ALERT with reason codes (Figure 2.1). The anomaly layer is optional by design so that the Phase III baseline can first be extended with rules and statistical models added once enough legitimate history exists. Only confirmed legitimate observations may update a profile, so alerted events never become training data automatically.

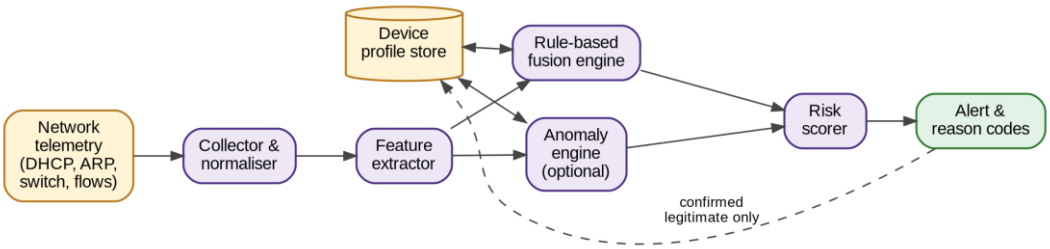


Figure 2.1 Overview of the proposed anomaly-based rogue-device detection system

2.3.2 OBJECTIVES OF THE PROPOSED SYSTEM

The proposed system is intended to:

- detect unauthorized devices even when they present a registered MAC address, by testing the consistency of location, fingerprint and behaviour;

- represent uncertainty through OK, REVIEW and ALERT states and explain every decision with signal-level reason codes;
- learn legitimate change through versioned, confirmed profile updates without being poisoned; and
- be evaluated by scenario and adversary tier with an ablation of each signal.

2.3.3 ADVANTAGES OF PROPOSED SYSTEM

Table 2.3 Proposed system compared with the existing MAC-whitelist control

Aspect	Existing MAC-whitelist control	Proposed multi-signal system
Evidence used	One field (MAC)	Identity, location, fingerprint, behaviour, time
Spoofed known MAC	Accepted (measured recall 0.00)	Port, fingerprint and behaviour mismatches raise risk
Output	Binary OK / ALERT	OK / REVIEW / ALERT with risk score
Explainability	Rule name only	Per-signal reason codes
Legitimate change	Manual list update	Versioned profiles with grace periods
Cost	Very low	Low: metadata only, open-source tools

In summary, the design removes the single point of failure created by trusting one identifier, keeps evidence transparent and auditable, avoids payload inspection, and allows staged adoption in which each added layer must justify itself against the preceding one.

CHAPTER 3
SYSTEM DESIGN

3.1 SYSTEM ARCHITECTURE

The architecture is modular and organized into six layers (Figure 3.1). The monitored network contains authorized endpoints, infrastructure, IoT systems and potential rogue devices. The observation and collection layers receive telemetry passively, through a mirrored (SPAN) port, so an experimental detector cannot disrupt production traffic. The normalization layer converts raw records into a common event schema, and the feature layer maintains rolling statistics such as packets, bytes and destinations per window. The detection layer combines deterministic checks, which give explainability, with optional machine-learning models. The scoring layer fuses their outputs and the alerting layer writes structured records for a dashboard or SIEM. Evidence collection is deliberately separated from decision logic, so the same detector can be evaluated on synthetic events, on Mininet traffic and later on production-like captures.

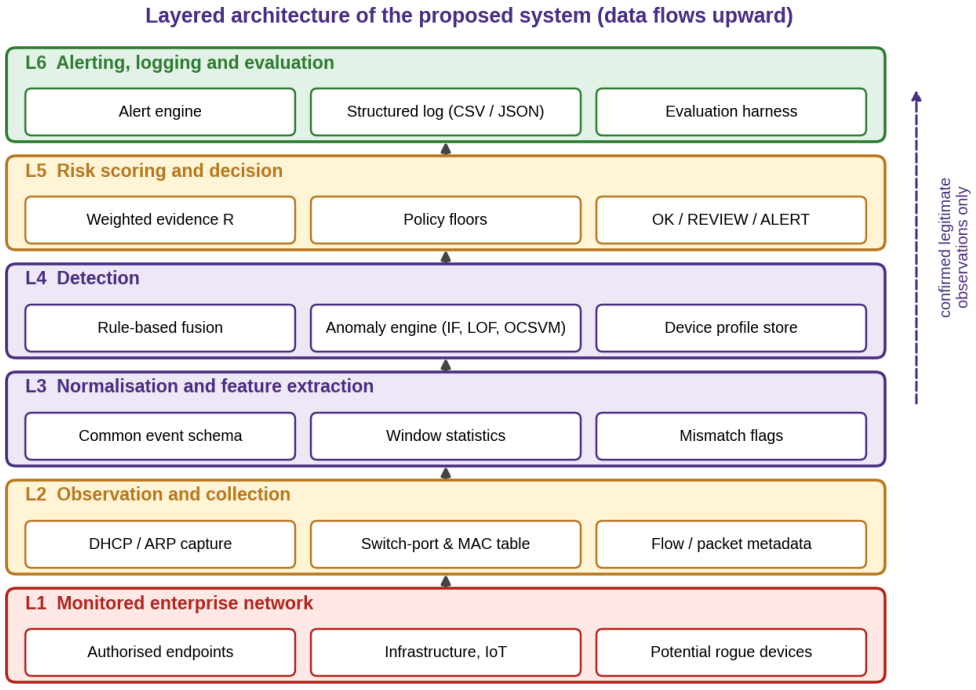


Figure 3.1 Layered system architecture of the proposed detector

The planned live testbed uses Mininet [23] with an Open vSwitch carrying three registered hosts (AA:BB:CC:00:00:01 to :03 on ports 1 to 3), a DHCP and gateway host, rogue hosts and a monitor host receiving mirrored traffic, observed with Scapy [24] and tshark. The prototype is lightweight; Tables 3.1 and 3.2 give the reference configuration.

Table 3.1 Hardware requirements (reference configuration)

Component	Specification	Purpose
Processor	Multi-core x86-64, 4 cores or more	Virtual hosts, capture and model training
Memory	8 GB RAM minimum (16 GB recommended)	Parallel hosts and feature windows
Storage	20 GB free (SSD preferred)	Captures, logs and datasets
Optional lab	Managed switch with SPAN, access point	Validation on real hardware

Table 3.2 Software requirements

Component	Technology	Role
Operating system	Linux (Ubuntu 22.04 LTS or later)	Mininet and packet tooling
Language and ML	Python 3, scikit-learn [20]	Implementation, IF / LOF / One-Class SVM
Emulation and capture	Mininet, Open vSwitch, Scapy, tshark	Live LAN testbed [23]
Storage and tracking	CSV (extensible to SQLite), Git	Logging and reproducibility

3.2 MODULE DESIGN

The system is split into nine modules with narrow interfaces (Figure 3.2 and Table 3.3), so the baseline detector can be replaced by the fusion detector without rewriting collection or evaluation. M1, M2, M9 and the baseline detector are the implemented Phase III slice; M3 to M8 are designed and specified here.

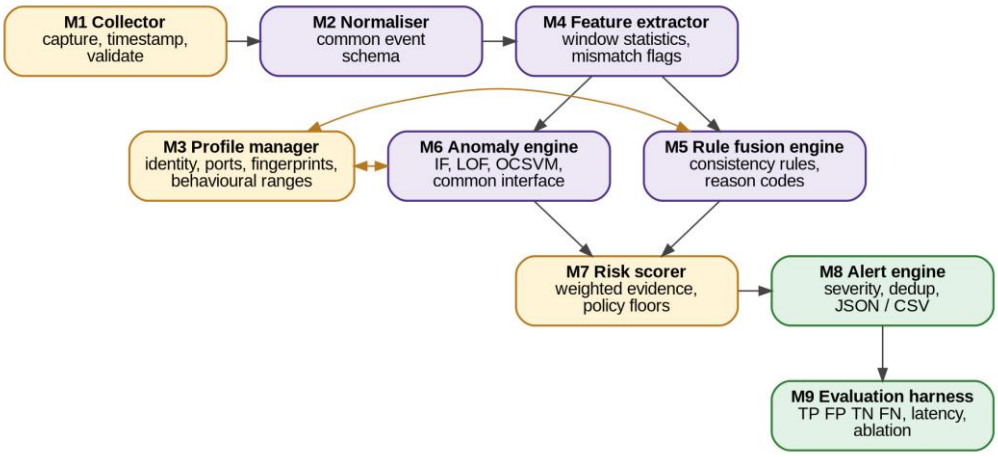


Figure 3.2 Module decomposition and interfaces

Table 3.3 Module interfaces and implementation status

ID	Module	Input / output	Status
M1	Collector	Raw packets or logs / timestamped event	Slice implemented
M2	Normaliser	Raw event / common-schema event	Slice implemented
M3	Profile manager	Confirmed events / versioned device profile	Designed
M4	Feature extractor	Event, window, profile / feature vector, flags	Designed
M5	Rule fusion engine	Flags, profile / signals and reason codes	Designed
M6	Anomaly engine	Feature vector / normalised anomaly score	Designed (optional)
M7	Risk scorer	Signals, score / risk R and decision	Designed
M8	Alert engine	Decision, evidence / alert record (CSV, JSON)	Designed
M9	Evaluation harness	Log with ground truth / TP, FP, TN, FN, P, R, F1, FPR, latency	Slice implemented

Profile manager and features. A device profile holds the identity set, expected switch ports, DHCP fingerprint distribution and normal traffic ranges, and each value is either confirmed, provisional, stale or retired. Learning is conservative: identity and location attributes change only after confirmation, behavioural ranges change by controlled decay, and a grace period accepts both old and new patterns during a legitimate transition. Features cover identity novelty, port novelty, fingerprint match, packets and bytes per minute, destination count and active hours. Missing data is never read as zero risk: absent telemetry is marked unavailable.

Anomaly engine. Isolation Forest, LOF and One-Class SVM sit behind one fit-and-score interface and are trained only on legitimate baseline windows, so rogue samples never enter the normal training set. The model version and preprocessing parameters are stored with every experiment.

Risk scorer. Heterogeneous evidence is combined as a transparent weighted sum of five components in the range 0 to 1: identity inconsistency I, location inconsistency L, fingerprint inconsistency F, behavioural anomaly B and temporal anomaly T.

$$R = w_I \cdot I + w_L \cdot L + w_F \cdot F + w_B \cdot B + w_T \cdot T \tag{3.1}$$

The weights are non-negative and sum to one; the illustrative starting values are w_I = 0.30, w_L = 0.25, w_F = 0.20, w_B = 0.20 and w_T = 0.05, to be validated by ablation and not tuned on the test set. R is an evidence score, not a calibrated probability. The behavioural component

uses a robust deviation from the device baseline, where MAD is the median absolute deviation and k (for example 3) sets the saturation point.

$$B = min(1, |x - median| / (k \cdot 1.4826 \cdot MAD)) \quad (3.2)$$

R below the review threshold gives OK, R from the review threshold up to the alert threshold gives REVIEW, and higher values give ALERT. The illustrative thresholds are 0.50 and 0.75 and must be fixed on validation data. A weighted sum can dilute strong but isolated evidence, so four policy floors guarantee a minimum decision: **PF1**, a registered identity on a port outside its allowed set is at least REVIEW; **PF2**, a registered identity with a fingerprint outside its profile is at least REVIEW; **PF3**, both a location and a fingerprint mismatch on a registered identity is ALERT; **PF4**, an unregistered identity is at least REVIEW. Table 3.4 gives six worked examples computed from Equation 3.1; they illustrate the scoring logic and are not experimental results.

Table 3.4 Worked examples of risk scoring with policy floors (illustrative)

ID	Situation	I, L, F, B, T	R	Final decision
E1	Registered MAC, expected port and fingerprint	0, 0, 0, 0.05, 0	0.010	OK
E2	Registered MAC on a new port only	0, 1, 0, 0.05, 0	0.260	REVIEW (PF1)
E3	Registered MAC, new port and new fingerprint (Phase III spoof pattern)	0, 1, 1, 0.05, 0	0.460	ALERT (PF3)
E4	Registered MAC, new fingerprint, unusual traffic	0, 0, 1, 0.45, 0	0.290	REVIEW (PF2)
E5	Unregistered MAC, ordinary traffic	1, 0, 0, 0.05, 0	0.310	REVIEW (PF4)
E6	Registered MAC, unusual traffic and timing only	0, 0, 0, 1, 1	0.250	OK, logged

Figure 3.3 places the six examples on the score scale.

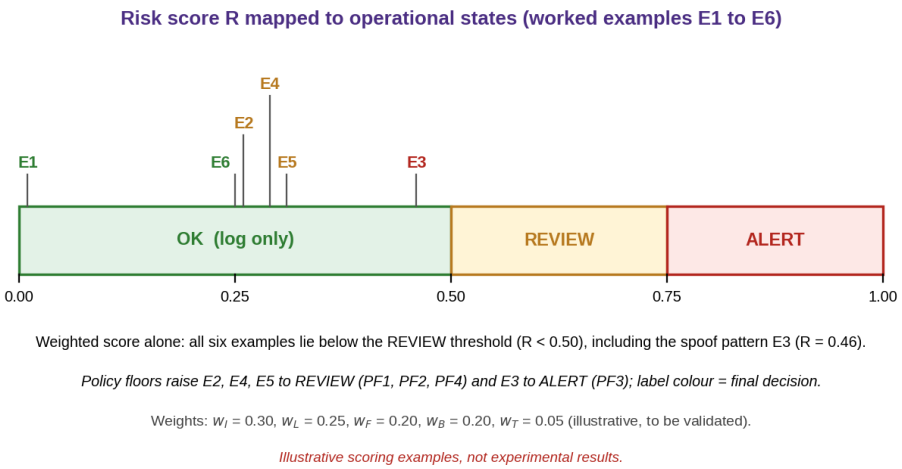


Figure 3.3 Risk score mapped to operational states with worked examples

Example E3 shows why floors are needed: the exact pattern of the Phase III spoofed events scores only 0.46 and would stay below REVIEW on score alone. Example E6 shows an honest limitation of the starting weights, since behaviour alone does not reach REVIEW; the design handles it with a persistence rule (a deviation sustained over several windows is escalated) and records it for the ablation study. The alert engine writes event identifier, time, identity, port, R, severity, reason codes and detector version, groups repeated alerts into one incident, and never exposes ground truth in operational mode.

3.3 DATA FLOW / UML DIAGRAMS

The level-1 data flow diagram (Figure 3.4) has five processes and three data stores. An observation is timestamped and validated, and invalid records are logged as collection-quality events and not silently dropped. After normalization the event is matched to a profile, deterministic comparisons run first because they are cheap and give immediate reason codes, and the anomaly score is added when enough history exists. The fusion stage computes R. Only confirmed legitimate observations flow back to the profile store (dashed arrow). The external entities are the network devices, the security administrator and the experiment controller.

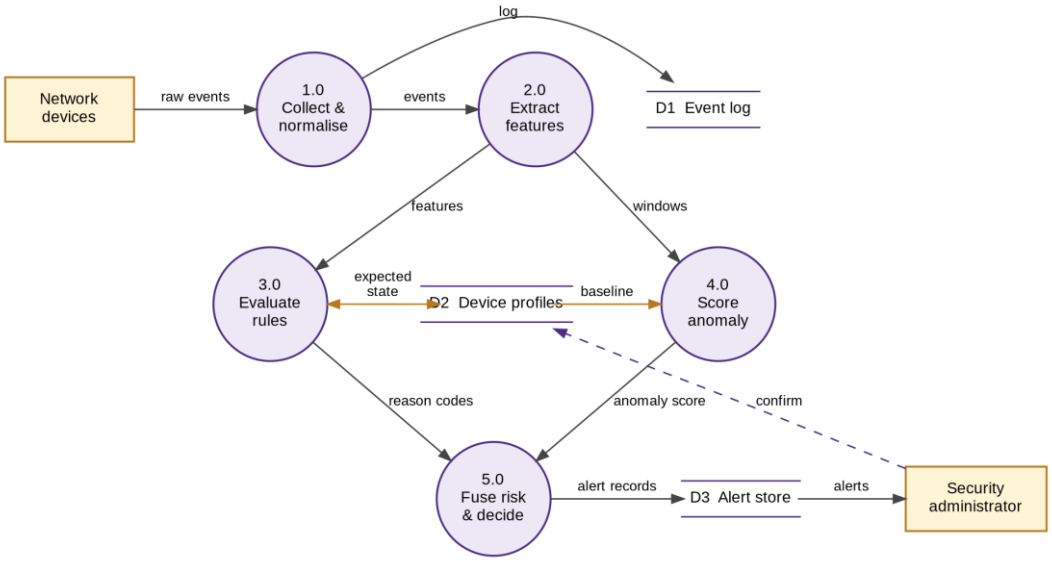


Figure 3.4 Level-1 data flow diagram

The use case diagram (Figure 3.5) shows four actors: the security administrator, who views alerts, confirms devices and configures thresholds; the experiment controller, who runs attack scenarios and requests metrics; the network devices, which are the passive source of observations; and a future SIEM or ticketing consumer.

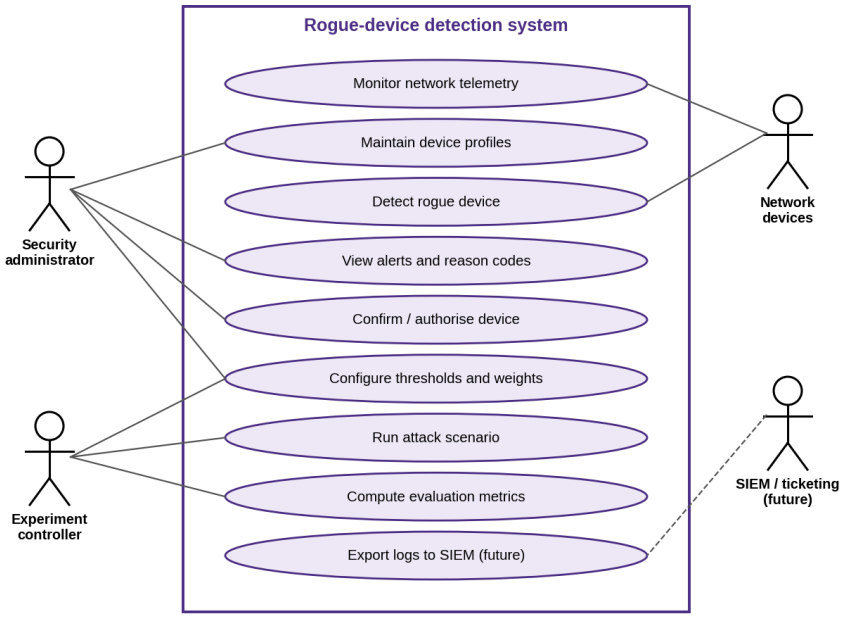


Figure 3.5 Use case diagram

The activity diagram (Figure 3.6) shows the decision procedure for one observation: validation, the registered-identity branch, the port, fingerprint and behaviour checks, the anomaly score when a baseline exists, fusion with the policy floors, and the three terminal

actions OK (log only), REVIEW (queue for the analyst) and ALERT (notify with reason codes).

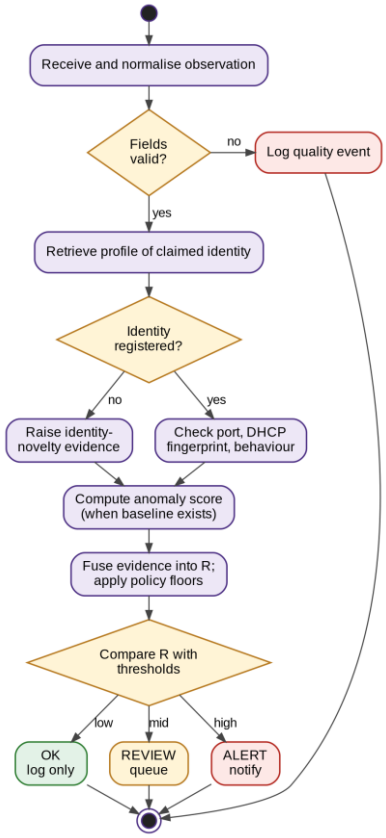


Figure 3.6 Activity diagram of the detection procedure for one observation

The sequence diagram (Figure 3.7) shows the messages exchanged between device, collector, feature extractor, profile store, detector, risk scorer, alert engine and administrator, ending with the optional confirmation of a legitimate device and the versioned profile update.

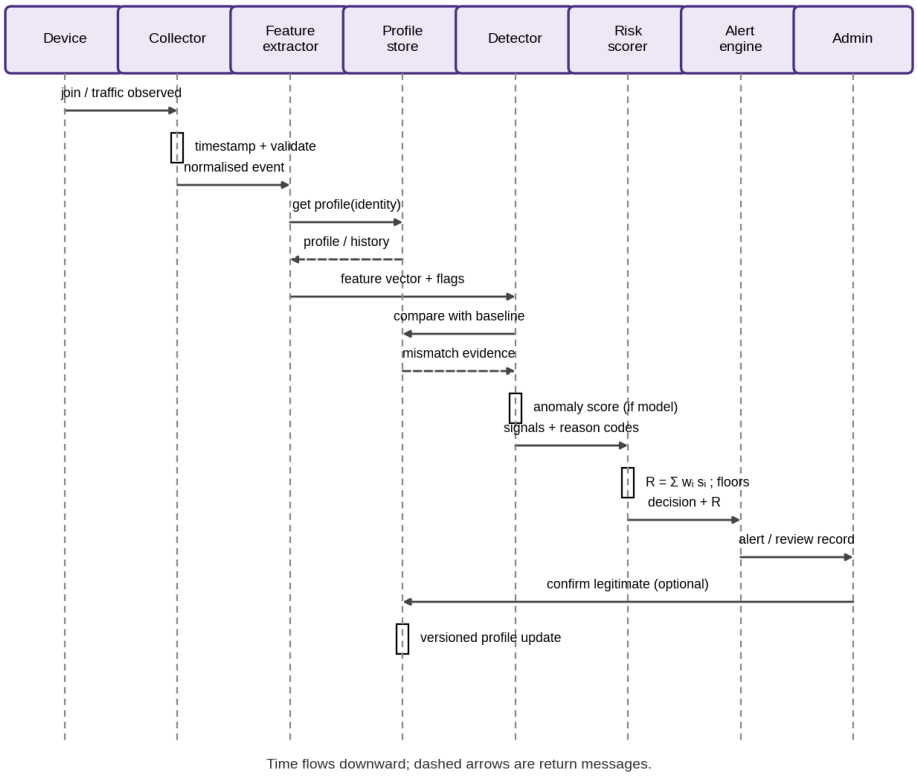


Figure 3.7 Sequence diagram for processing one device observation

3.4 THREAT MODEL AND ATTACK SURFACE

The assets protected are the device inventory view, the profile store, the logs and alerts, and the confidentiality of telemetry. The adversary can connect a device to the monitored environment, change software-visible identifiers (MAC address, hostname, DHCP client options) and may infer the detector's inputs and imitate selected signals. The adversary is not assumed to compromise the monitoring host, the switch logs or the DHCP server, or to defeat cryptographic authentication. Figure 3.8 shows the resulting trust zones and boundaries.

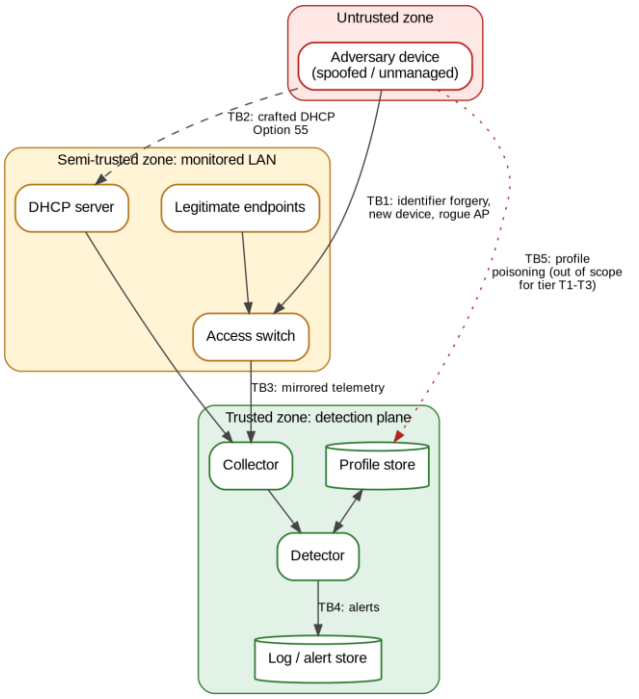


Figure 3.8 Trust zones and trust boundaries (TB1 to TB5)

To avoid evaluating the detector only against the assumptions used to design it, the attacker is given progressively more capability. In **T1** only the MAC address is copied; in **T2** the DHCP fingerprint is also reproduced; in **T3** network location is also controlled or imitated. The Phase III spoofed workload corresponds to T1, and Figure 3.9 shows which evidence each tier reproduces.

Adversary tiers: evidence the attacker reproduces (red) and evidence that remains independent (green)

	T1 naive spoof	T2 protocol mimic	T3 location-aware
MAC address	imitated	imitated	imitated
IP / lease	independent	imitated	imitated
DHCP Option 55	independent	imitated	imitated
Switch port / location	independent	independent	imitated
Traffic behaviour	independent	independent	independent
Timing / periodicity	independent	independent	independent

Residual evidence shrinks from T1 to T3: the evaluation measures how detection degrades, not only whether it succeeds.

Figure 3.9 Adversary tiers and the evidence each tier reproduces

Ground truth is established independently of the detector: the experiment controller knows when each rogue device is introduced and removed. The five scenarios are an unauthorized laptop, a rogue access point (which creates several new identities), an unmanaged IoT device

(sparse, periodic traffic), a personal hotspot (limited visibility) and a MAC-spoofed device. Table 3.5 lists the threats against the system itself using the STRIDE categories [25].

Table 3.5 Threat enumeration (STRIDE checklist)

ID	Category	Threat	Primary mitigation
T-1	Spoofing	Rogue copies a registered MAC and DHCP options	Port history, fingerprint, behaviour, PF1 to PF3
T-2	Tampering	Profiles edited to normalise attacker behaviour	Confirmed-only, versioned updates
T-3	Tampering	Logs altered after the fact	Restricted write access, hashes, append-only log
T-4	Information disclosure	Telemetry reveals user activity	Metadata only, access control, retention limits
T-5	Denial of service	Flood of new identities overwhelms analysts	Rate limiting and incident grouping
T-6	Evasion	Low-and-slow behaviour stays within limits	Multi-window persistence, identity and location evidence

3.5 SECURITY CONTROLS

The detector is itself a sensitive component, since an attacker who can rewrite evidence or poison profiles can undermine the whole process. Table 3.6 maps the controls to the requirements of Section 1.5 and the threats of Table 3.5.

Table 3.6 Security controls and how they are verified

Control	Requirement	Verification	Mapping
Log integrity	Restricted write access; audit trail; hashes of closed logs	Review permissions, compare hashes	SR-3, T-3
Profile integrity	Only confirmed, versioned updates	Review profile-change log	SR-3, SR-4, T-2
Metadata only	No payload capture or decryption	Inspect capture filters	SR-5, T-4
Fail-safe handling	Missing telemetry recorded as uncertainty	Inject missing-signal cases	SR-6, T-1
Least privilege, passive mode	Packet-read rights only; mirrored port; no blocking	Inspect permissions and switch setup	SR-7, T-5
Configuration control	Version-controlled configuration tied to a run ID	Compare run ID with commit	Reproducibility

The controls follow defence in depth: preventive controls (confirmed-only updates, least privilege, passive deployment), detective controls (integrity hashes, audit logs, reason codes) and process controls (frozen test set, version-controlled configuration, documentation of failed experiments) that protect the integrity of the research results.

CHAPTER 4
SYSTEM DESCRIPTION

4.1 DATA COLLECTION

Two data sources are used in sequence. The first, implemented in Phase III, is a synthetic event generator, valuable because every record has a known ground truth and the workload is exactly reproducible. The second, planned, is a Mininet-emulated LAN in which Scapy and tshark observe genuine ARP and DHCP traffic. Synthetic data verifies software logic but cannot reproduce every property of real protocol timing, so it is treated as a controlled baseline and not a substitute for production telemetry. The relevant sources are DHCP exchanges (Option 55 and lease data), ARP traffic, the switch port and MAC table, and flow or header metadata; payload content is avoided. The report also states the observation boundary: a device not visible at the monitoring point is not necessarily absent, which matters for personal hotspots and segmented networks.

Every observation is stored with the fields in Table 4.1. The label fields exist for evaluation only and are never passed to a detector as input; otherwise reported accuracy would be meaningless.

Table 4.1 Event dataset schema

Field	Type	Example	Purpose
timestamp	datetime	2026-09-20T09:00:00	Ordering and latency
mac	string	AA:BB:CC:00:00:01	Claimed identity
ip, switch_port	string, integer	192.168.1.10, 1	Network context and location
dhcp_fingerprint	string	1,3,6,15,119,252	Protocol fingerprint (Option 55)
scenario, is_rogue	label	legit / spoofed_mac, False / True	Ground truth (evaluation only)

The Phase III generator uses a fixed seed (42), three registered devices with fixed MAC, port and DHCP fingerprint, and 25 events spaced 30 seconds apart from 09:00:00 on 20 September 2026. Twenty are legitimate; five reuse a registered MAC on a different port (legitimate port plus ten), with a new IP in 192.168.1.200 to .204 and the fingerprint 1,15,3,6. The composition in the executed log is shown in Table 4.2 and the distribution of the events over time in Figure 4.1.

Table 4.2 Composition of the Phase III workload (from the executed log)

Registered device	Port	Legitimate	Rogue	Registered DHCP fingerprint
AA:BB:CC:00:00:01	1	11	2	1,3,6,15,119,252
AA:BB:CC:00:00:02	2	2	0	1,3,6,15,119,252
AA:BB:CC:00:00:03	3	7	3	1,121,3,6,15,119
Total		20	5	

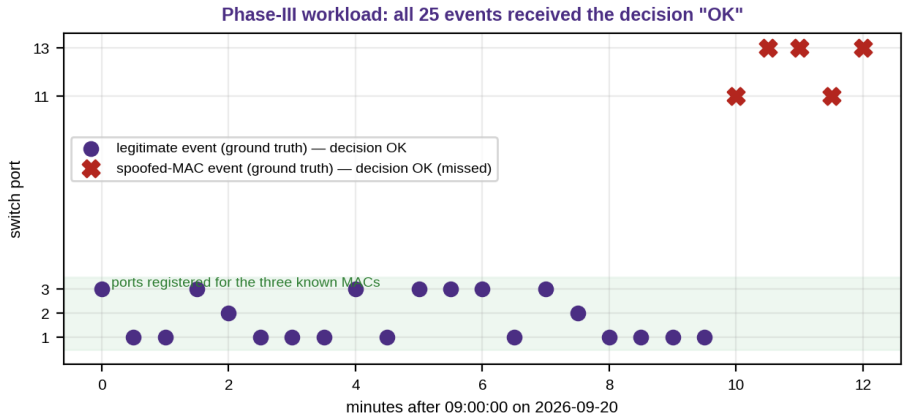


Figure 4.1 Phase III workload: switch port of each event over time and the decision the baseline gave

Two properties limit how the workload may be interpreted. The random draw gave an uneven legitimate mix (11, 2 and 7 events), and every rogue event differs from the profile in IP, port and fingerprint at once, with one constant fingerprint. A multi-signal detector will therefore separate it trivially: the workload is a sanity check of the pipeline and of the baseline failure mode, not a test of robustness, which is what tiers T2 and T3 are for. Data-quality checks (missing fields, invalid addresses, duplicates, one MAC on two ports at once, label consistency) log a collection-quality event and never silently drop a record.

4.2 DATA PREPROCESSING

Splitting must be time aware: training on earlier legitimate history, selecting thresholds on a later validation period and testing on a final period left untouched until the end. Random mixing of adjacent windows leaks device-specific patterns and inflates results. Features fall into eight families: identity (first seen, in inventory), location (port novelty), protocol (DHCP fingerprint match), traffic volume, communication (destination count, port diversity), temporal (active hours, inter-arrival), historical deviation (robust z-score against the device baseline) and context (maintenance windows), as summarized in Figure 4.2.

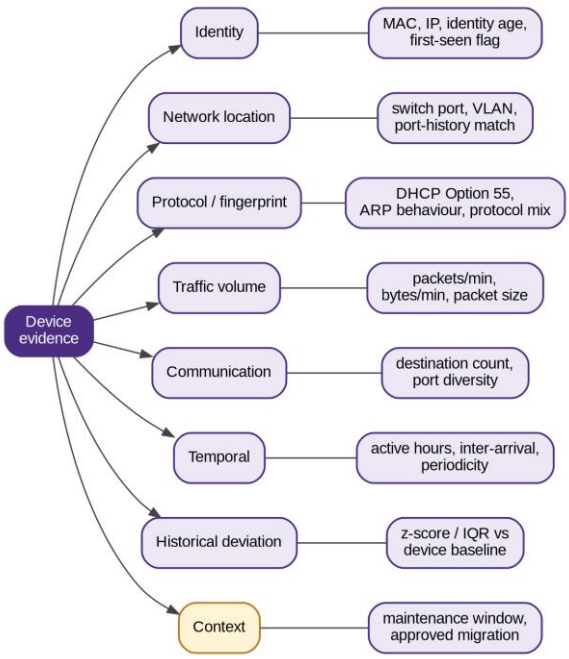


Figure 4.2 Feature taxonomy used by the framework

Features are normalised because their scales differ widely: Boolean flags are used as 0/1, skewed counts use log(1 + x) with robust scaling, cyclical hour of day uses sine and cosine, and anomaly scores from different models are rescaled to [0, 1] on validation data. Parameters are learned on baseline data only and applied unchanged to validation and test data, since fitting on the full dataset leaks test information. A missing DHCP fingerprint is not a mismatch; the signal is marked unavailable and the confidence of the decision is lowered (requirement SR-6).

4.2.1 ALGORITHMS / METHODOLOGY

The method is incremental: a deterministic baseline, then added network and behavioural features, then unsupervised models on the same feature representation, and finally the fused risk score. If the final detector improves, the gain can be attributed to added evidence and not a different environment; if machine learning brings no gain, the result is still useful because it shows the complexity is unjustified. Fixed seeds, configuration files and run identifiers make each experiment repeatable.

Baseline (MAC whitelist). A set of three known MACs; a known address gives OK and any other gives ALERT. It ignores port, IP, fingerprint and traffic. The source is in Appendix A.

Rule-based fusion. The detector retrieves the profile of the claimed identity, emits location, fingerprint and identity signals, and applies Equation 3.1 and the floors PF1 to PF4. Configurable weights allow the ablation D0 to D6 in Table 4.3.

Unsupervised models. Isolation Forest builds random trees and scores points that are isolated in few splits as anomalous [17]; Local Outlier Factor flags points whose local density is much lower than that of their neighbours [18]; One-Class SVM learns a boundary around normal data in a kernel space [19]. All three train on legitimate windows only, with parameters (contamination, n_neighbors, nu and gamma) chosen on validation data. They are compared under identical preprocessing, labels and partitions, per scenario and tier.

Threshold selection. Thresholds are chosen by inspecting score distributions on the validation set, fixing a maximum acceptable false-positive rate and measuring recall, freezing the values before testing, and reporting sensitivity at several thresholds. Two thresholds preserve information without forcing a binary decision.

Metrics. With TP, FP, TN and FN as usual, the metrics are

$$Precision = TP / (TP + FP), \quad Recall = TP / (TP + FN) \tag{4.1}$$

$$F1 = 2 \cdot Precision \cdot Recall / (Precision + Recall), \quad FPR = FP / (FP + TN) \tag{4.2}$$

Detection latency is the time from the first rogue observation to the first qualifying alert, reported as mean, median and worst case. Recall matters most because a missed rogue is a direct security failure, but recall alone is insufficient since a detector that alerts on everything has perfect recall, so precision, F1 and FPR are reported with it [26, 27].

4.2.2 RESULTS AND DISCUSSION

The completed experiment is the Phase III baseline: 25 synthetic events (20 legitimate, 5 rogue), three profiles and a fixed seed. Each event is processed, one row is written to a CSV log with the decision and ground truth, and the evaluator reconstructs TP, FP, TN and FN from that log. Re-executing the submitted code gives **TP = 0, FP = 0, TN = 20 and FN = 5**. Precision, recall and F1-score are 0.00 and the false-positive rate is 0.00. The detector raised no alert at all: all 25 events received the decision OK (Figure 4.3).

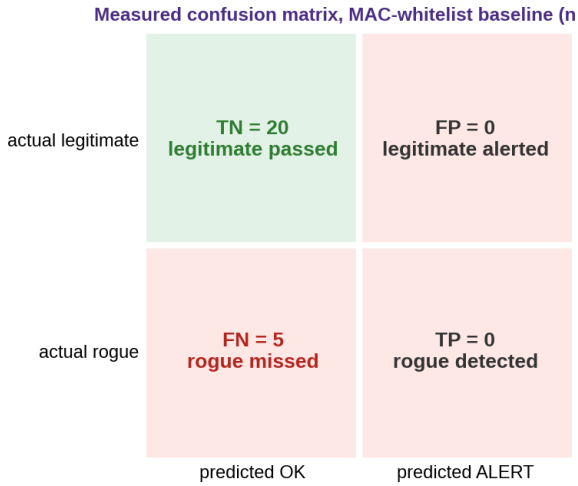


Figure 4.3 Measured confusion matrix of the MAC-whitelist baseline (n = 25)

Accuracy, the share of all events classified correctly, is (0 + 20) / 25 = 0.80, which looks acceptable although every rogue event was missed, and the zero false-positive rate arises only because no alert was ever produced. A detector that never alerts can look accurate on an imbalanced workload, which is why recall, precision, F1 and FPR are the headline metrics. Figure 4.4 shows why the baseline failed: in all five spoofed events the MAC is identical, the only field it reads, while IP, port and DHCP fingerprint all differ. The information needed to detect the events was present but unused, which is the evidential basis for the multi-signal design.

What differs from the registered profile in each of the five spoofed events

Event	MAC	IP	Switch port	DHCP Option 55	MAC-only decision
R1 09:10	same	different	different	different	OK (missed)
R2 09:10	same	different	different	different	OK (missed)
R3 09:11	same	different	different	different	OK (missed)
R4 09:11	same	different	different	different	OK (missed)
R5 09:12	same	different	different	different	OK (missed)

Three independent fields differ in every rogue event; the baseline reads only the one field that does not.

Figure 4.4 Fields that differ from the registered profile in each spoofed event (executed log)

The next experiment runs identical events through successive configurations to isolate each signal (Table 4.3), repeated across T1, T2 and T3. Numeric entries other than the measured baseline are marked TBM (to be measured) and no value is estimated in advance. On the present workload, any detector that reads port or fingerprint will separate the rogue events,

so strong D1 to D3 numbers there only confirm correct implementation; the informative results will come from T2, T3 and an expanded dataset with legitimate negative controls.

Table 4.3 Detector configurations for the ablation study and result template

ID	Signals enabled	Precision	Recall	F1	FPR	Latency
D0	MAC only (measured, T1)	0.00	0.00	0.00	0.00	n/a
D1	MAC + port	TBM	TBM	TBM	TBM	TBM
D2	MAC + DHCP fingerprint	TBM	TBM	TBM	TBM	TBM
D3	MAC + port + DHCP	TBM	TBM	TBM	TBM	TBM
D4	D3 + behaviour + time	TBM	TBM	TBM	TBM	TBM
D5	Each anomaly model alone	TBM	TBM	TBM	TBM	TBM
D6	D4 + anomaly score (full)	TBM	TBM	TBM	TBM	TBM

False positives (a laptop moving to an approved but unseen port, an update changing the DHCP request list) cause alert fatigue and are treated as a profile-management problem, addressed by grace periods and role-aware profiles and not only by raising thresholds. False negatives are analysed by scenario and tier, since missing every T3 spoof is a different limitation from missing every low-volume IoT device.

The evidence has clear limits: 25 events, synthetic traffic, only T1 measured, one device with just two legitimate events, and no execution yet of the fusion or anomaly evaluations. The result is therefore a proof of concept. It supports one narrow but important conclusion: a MAC-only whitelist is structurally incapable of detecting a device that presents a trusted MAC. It does not justify any claim that Isolation Forest, LOF or One-Class SVM is superior. The proposed architecture is a progression from static identity to contextual identity to behavioural anomaly detection, and each stage must justify its complexity against the previous one.

CHAPTER 5
CONCLUSION

This report presented an anomaly-based framework for detecting rogue devices in corporate environments. Its central premise is that unauthorized-device detection should not depend on static identifiers alone. Because corporate networks contain legitimate variation, mobility, dynamic addressing and growing IoT populations, a practical detector must combine identity evidence with contextual and behavioural observations. The work set out a layered architecture, nine modules with defined interfaces, a risk score with explicit policy floors, a threat model with adversary tiers T1 to T3, security requirements and controls, and an evaluation methodology with time-aware splits and signal ablation.

The completed Phase III baseline provides the experimental reference. Re-executing the submitted code on the 25-event workload reproduces TP = 0, FP = 0, TN = 20 and FN = 5: the MAC whitelist accepted every legitimate event and missed every spoofed one, with precision, recall and F1-score of 0.00 and an accuracy of 0.80 that conceals the failure. The executed log also shows that IP address, switch port and DHCP fingerprint differed from the registered profile in all five missed events. The principal contribution is therefore the framework and the evaluation methodology, not an unsupported claim of production-level accuracy; measured results are reported as measured, and the performance of the multi-signal and anomaly models is reported as planned until those experiments are run.

Limitations. The workload is small and synthetic, only the T1 baseline was measured, the live Mininet testbed and the fusion detector were designed but not evaluated, the legitimate mix is uneven, and the illustrative risk weights have not been validated.

Future work. (1) Replace the 25-event workload with repeated laboratory captures covering several device roles and legitimate changes. (2) Build the Mininet testbed, implement the MAC + port and MAC + port + DHCP fusion detectors, and add the T2 and T3 adversaries. (3) Collect behavioural history and compare Isolation Forest, LOF and One-Class SVM with fixed splits. (4) Forward structured alerts to a SIEM and introduce quarantine through NAC only after false-positive behaviour is understood. (5) Study graph-based device relationships for rogue access points, adversarial profile poisoning, and the privacy and governance of the collected metadata.

REFERENCES

[1] Rose, S., Borchert, O., Mitchell, S., and Connelly, S. (2020). *Zero Trust Architecture*. NIST Special Publication 800-207.

[2] Plummer, D. C. (1982). *An Ethernet Address Resolution Protocol*. RFC 826, Internet Engineering Task Force.

[3] Droms, R. (1997). *Dynamic Host Configuration Protocol*. RFC 2131, Internet Engineering Task Force.

[4] Chandola, V., Banerjee, A., and Kumar, V. (2009). "Anomaly detection: A survey." *ACM Computing Surveys*, 41(3), Article 15.

[5] IEEE. (2020). *IEEE Standard for Local and Metropolitan Area Networks – Port-Based Network Access Control*, IEEE Std 802.1X-2020.

[6] United Nations. (2015). *Transforming our world: The 2030 Agenda for Sustainable Development*. A/RES/70/1.

[7] Scarfone, K. and Mell, P. (2007). *Guide to Intrusion Detection and Prevention Systems (IDPS)*. NIST Special Publication 800-94.

[8] Aboba, B., Blunk, L., Vollbrecht, J., Carlson, J., and Levkowetz, H. (2004). *Extensible Authentication Protocol (EAP)*. RFC 3748, Internet Engineering Task Force.

[9] Rigney, C., Willens, S., Rubens, A., and Simpson, W. (2000). *Remote Authentication Dial In User Service (RADIUS)*. RFC 2865, Internet Engineering Task Force.

[10] Alexander, S. and Droms, R. (1997). *DHCP Options and BOOTP Vendor Extensions*. RFC 2132, Internet Engineering Task Force.

[11] Miettinen, M., Marchal, S., Hafeez, I., Asokan, N., Sadeghi, A.-R., and Tarkoma, S. (2017). "IoT Sentinel: Automated device-type identification for security enforcement in IoT." *Proceedings of IEEE ICDCS*, 2177–2184.

[12] Sivanathan, A., Gharakheili, H. H., Loi, F., Radford, A., Wijenayake, C., Vishwanath, A., and Sivaraman, V. (2018). "Classifying IoT devices in smart environments using network traffic characteristics." *IEEE Transactions on Mobile Computing*, 18, 1745–1759.

[13] Bezawada, B., Bachani, M., Peterson, J., Shirazi, H., Ray, I., and Ray, I. (2018). "Behavioral fingerprinting of IoT devices." *Proceedings of the ACM Workshop on Attacks and Solutions in Hardware Security (ASHES)*, 41–50.

[14] Liu, Y., Wang, J., Li, J., Niu, S., and Song, H. (2021). "Machine learning for the detection and identification of Internet of Things (IoT) devices: A survey." *IEEE Internet of Things Journal*.

[15] Meidan, Y., Bohadana, M., Mathov, Y., Mirsky, Y., Breitenbacher, D., Shabtai, A., and Elovici, Y. (2018). "N-BaIoT: Network-based detection of IoT botnet attacks using deep autoencoders." *IEEE Pervasive Computing*, 17, 12–22.

[16] Mirsky, Y., Doitshman, T., Elovici, Y., and Shabtai, A. (2018). "Kitsune: An ensemble of autoencoders for online network intrusion detection." *Proceedings of the Network and Distributed System Security Symposium (NDSS)*.

[17] Liu, F. T., Ting, K. M., and Zhou, Z.-H. (2008). "Isolation Forest." *Proceedings of the IEEE International Conference on Data Mining (ICDM)*, 413–422.

[18] Breunig, M. M., Kriegel, H.-P., Ng, R. T., and Sander, J. (2000). "LOF: Identifying density-based local outliers." *Proceedings of the ACM SIGMOD International Conference on Management of Data*, 93–104.

[19] Schölkopf, B., Platt, J. C., Shawe-Taylor, J., Smola, A. J., and Williamson, R. C. (2001). "Estimating the support of a high-dimensional distribution." *Neural Computation*, 13(7), 1443–1471.

[20] Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). "Scikit-learn: Machine learning in Python." *Journal of Machine Learning Research*, 12, 2825–2830.

[21] Alani, M. M., Awad, A. I., and Barka, E. (2023). "ARP-PROBE: An ARP spoofing detector for Internet of Things networks using explainable deep learning." *Internet of Things*, 23, 100861.

[22] Wang, Y., Rimal, B. P., Elder, M., Crespo Maldonado, S. I., Chen, H., Koball, C., and Ragothaman, K. (2022). "IoT device identification using supervised machine learning." *Proceedings of the IEEE International Conference on Consumer Electronics*.

[23] Lantz, B., Heller, B., and McKeown, N. (2010). "A network in a laptop: Rapid prototyping for software-defined networks." *Proceedings of HotNets-IX*.

[24] Biondi, P. and the Scapy community. *Scapy: Interactive packet manipulation program*. https://scapy.net.

[25] Howard, M. and Lipner, S. (2006). *The Security Development Lifecycle*. Microsoft Press.

[26] Fawcett, T. (2006). "An introduction to ROC analysis." *Pattern Recognition Letters*, 27(8), 861–874.

[27] Saito, T. and Rehmsmeier, M. (2015). "The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets." *PLoS ONE*, 10(3), e0118432.

APPENDIX

APPENDIX A – SOURCE CODE

The source code developed for the Rogue Device Detection System consists of Python modules for generating network events, detecting suspicious devices, processing detection results, and evaluating system performance. The event generator creates network events containing MAC address, IP address, switch port, DHCP fingerprint, timestamp, and ground-truth status. The detection module performs the initial MAC-based verification, while the evaluation module calculates detection performance using TP, TN, FP, and FN values.

The proposed system can further integrate feature extraction, device profiling, rule-based fusion, anomaly detection, risk scoring, and alert generation to improve detection against MAC-spoofed devices.

APPENDIX B – TEST DATA AND RESULTS

The test dataset contains 25 network events, consisting of 20 legitimate devices and 5 rogue-device events. The rogue events use trusted MAC addresses but contain changes in IP address, switch-port information, and DHCP fingerprint. This dataset is used to demonstrate the limitation of MAC-address-only detection.

The baseline MAC-whitelist detector classified all 25 events as legitimate. The resulting performance was:

- True Positive (TP): 0
- True Negative (TN): 20
- False Positive (FP): 0
- False Negative (FN): 5
- Accuracy: 80%
- Precision: 0
- Recall: 0
- F1-Score: 0

These results demonstrate that relying only on MAC addresses is insufficient for detecting devices that spoof trusted identities.

APPENDIX C – SAMPLE ROGUE DEVICE EVENT

MAC Address: AA:BB:CC:00:00:01

IP Address: 192.168.1.200

Switch Port: 11

DHCP Fingerprint: 1,15,3,6

Expected Status: ROGUE

Baseline Detection: OK

The above event represents a rogue device using a trusted MAC address while appearing on an unexpected switch port with a different DHCP fingerprint.

APPENDIX D – PROPOSED SYSTEM OUTPUT

The proposed system output contains the following information for each detected device:

- Device MAC Address
- IP Address
- Switch Port
- DHCP Fingerprint
- Device Profile
- Behavioural Information
- Risk Score
- Detection Status
- Reason for Alert
- Timestamp

LIST OF ABBREVIATIONS

ABBREVIATION	EXPANSION
ARP	Address Resolution Protocol
BYOD	Bring Your Own Device
DHCP	Dynamic Host Configuration Protocol
EAP	Extensible Authentication Protocol
F1	Harmonic mean of precision and recall
FN	False Negative
FP	False Positive
FPR	False Positive Rate
IF	Isolation Forest
IoT	Internet of Things
LAN	Local Area Network
LOF	Local Outlier Factor
MAC	Media Access Control
MAD	Median Absolute Deviation
ML	Machine Learning
NAC	Network Access Control
NIC	Network Interface Controller
OCSVM	One-Class Support Vector Machine
PF	Policy Floor
RADIUS	Remote Authentication Dial-In User Service
RQ	Research Question
SDG	Sustainable Development Goal
SIEM	Security Information and Event Management
SPAN	Switched Port Analyzer (port mirroring)
SVM	Support Vector Machine
TBM	To Be Measured
TN	True Negative
TP	True Positive
VLAN	Virtual Local Area Network